Numerical Methods: Eigenvalue Assembly and the Shooting Solver

Two routes to the same compressible boundary-layer eigenvalue — the global spectral (Chebyshev collocation) assembly and the marching (shooting) solver of `pyMack`

Personal reference note

July 1, 2026

Abstract

This note documents the two numerical machines that `pyMack` uses to extract a boundary-layer stability eigenvalue from the linearised disturbance equations. **Part A** is the *spectral* route: Chebyshev collocation turns the wall-normal derivative into a dense matrix, the four disturbance equations are assembled into one $4n \times 4n$ generalised eigenvalue problem $A\boldsymbol{\phi} = cB\boldsymbol{\phi}$ (the full block structure, all sixteen A-blocks, the five B-blocks, and the boundary-condition row indices are given explicitly), and a single call to a dense eigensolver returns *all* eigenvalues at once — ideal for surveying the spectrum and sweeping neutral curves and N-factors. **Part B** documents the *shooting* route (`pymack/temporal_shooting.py`): the *same* temporal problem is recast as a small first-order system, integrated from the freestream to the wall by Runge–Kutta with re-orthonormalisation, and the eigenvalue is pinpointed by driving a 3×3 wall matrix singular. Both return the same physical phase speed c. Every equation, coefficient, and default is transcribed from the code, cited by file, so the document doubles as a code↔math map. The spectral prose, equations, and figures are reproduced from the companion methodology note (verified line-by-line against the solver); the shooting material is written fresh from the source.

Contents

How to read this. Part A and Part B are two independent solvers for the *same* eigenvalue; either can be read alone. Boxes marked **"What problem is solved here"** state, at each stage, exactly which equation, which boundary conditions, and which unknown (the eigenvalue) are involved. *In plain words* notes give the intuition; grey Remark asides hold secondary detail. The physical formulation this document discretises — the mean flow, the linearisation, and the four disturbance equations $\hat{q} = (\hat{u}, \hat{v}, \hat{T}, \hat{p})$ — is developed in the companion Disturbance Equations note and summarised in the notation table below.

Notation. Overbars are mean (base-flow) quantities; hats are complex disturbance amplitudes. The continuous wall-normal amplitude is $\hat{q}(y) = (\hat{u}, \hat{v}, \hat{T}, \hat{p})$; its discrete (stacked, grid-sampled) form is the vector $\boldsymbol{\phi}$. A prime on a mean quantity is $\mathrm{D} \equiv \mathrm{d}/\mathrm{d}y$.

Base flow & transport		**Disturbance / eigenvalue**	
$\overline{U}, \overline{T}, \overline{\rho}{=}1/\overline{T}$	mean velocity, temp., density	$\hat{q}$; $\boldsymbol{\phi}$	amplitude (cont.; discrete)
$\overline{\mu}, \overline{\kappa}$	viscosity, conductivity	$\hat{u}, \hat{v}, \hat{T}, \hat{p}$	velocity/temp./pressure amplitudes
$C{=}\overline{\mu}/\overline{T}$	Chapman–Rubesin group	$\alpha,\ \omega$	wavenumber, frequency
$M_e,\ Re,\ Pr,\ \gamma$	Mach, Reynolds, Prandtl, c_p/c_v	$c{=}\omega/\alpha$	complex phase speed; c_r, c_i its parts
L^*	Mack length $\sqrt{\nu_e x/U_e}$	$\omega_i{=}\alpha c_i$	temporal growth rate
$R{=}\sqrt{Re_x}$	Mack Reynolds number	$\sigma{=}-\,\mathrm{Im}(\alpha)$	spatial growth rate
$\nu_R{=}1/(Re\overline{\rho})$	viscous prefactor	$N,\ n{=}N{+}1$	collocation points; grid size

Operator shorthands: D_1, D_2 derivative matrices; $\nu_R{=}1/(Re\overline{\rho})$; $\mathcal{C}_\kappa, \mathcal{C}_d, d_R$ conduction/dissipation prefactors; $\mathcal{L}_c$ conduction operator; A, B (temporal) and C_0, C_1, C_2 (spatial) operator blocks; Y, $M(c)$, $\sigma_{\min}(c)$ (Part B) shooting state, wall matrix, and its smallest singular value.

Part A — The spectral (Chebyshev collocation) method

The disturbance equations are four coupled ODEs in y with known base-flow coefficients but no closed-form solution. Part A converts them into a finite matrix, assembles the eigenvalue problem, and reads off the growth rate; it then sweeps the solve into neutral curves and N-factors.

1 Chebyshev collocation, from scratch

What problem is solved here. Replace the wall-normal derivative D by a finite matrix so the disturbance ODEs become linear algebra; each continuous amplitude $\hat{q}(y)$ becomes a vector of its values at chosen grid points.

Why Chebyshev? For smooth functions a polynomial interpolant through Chebyshev points converges *exponentially*, so few points give many digits. The Chebyshev–Gauss–Lobatto (GLL) nodes on $[-1, 1]$,

$$\xi_j = \cos\Big(\frac{\pi j}{N}\Big), \quad j = 0, 1, \ldots, N, \tag{1}$$

cluster near the ends $\xi = \pm 1$. **Node ordering matters:** $\xi_0 = +1$ and $\xi_N = -1$, so j runs from $+1$ down to -1.

The differentiation matrix. One polynomial of degree $\le N$ passes through the values $\{f(\xi_j)\}$; differentiating and evaluating at the nodes is linear, $f'(\xi_j) \approx \sum_k D_{jk} f(\xi_k)$, with (Don & Solomonoff; `spectral.py:chebyshev_D`)

$$D_{jk} = \frac{c_j}{c_k}\frac{(-1)^{j+k}}{\xi_j - \xi_k} \ (j \neq k), \qquad D_{jj} = -\sum_{k \neq j} D_{jk}, \qquad c_0 = c_N = 2, \ c_{\text{else}} = 1. \tag{2}$$

The diagonal is set so each row sums to zero (a constant function has zero slope); $c_0 = c_N = 2$ accounts for the doubled weight of the two endpoints.

In plain words. *D turns "the list of function values" into "the list of slope values." It is dense — every node's slope uses every other node's value — which is why the final stability matrices are dense.*

A 3-point worked example. For $N = 2$, $\xi = (1, 0, -1)$ and (2) gives

$$D = \begin{bmatrix} 1.5 & -2 & 0.5 \\ 0.5 & 0 & -0.5 \\ -0.5 & 2 & -1.5 \end{bmatrix}, \qquad f'(\xi_1) \approx \tfrac{1}{2} f(\xi_0) - \tfrac{1}{2} f(\xi_2) \text{ (centred difference).} \tag{3}$$

The second derivative on $[-1, 1]$ is the matrix square, $D_\xi^2 = D_\xi D_\xi$ (*not* the physical D_1 squared — see below).

Mapping to the physical layer. The flow lives on $y \in [0, y_{\max}]$ with points clustered at the *wall*. The algebraic map (Malik 1990; `spectral.py:map_domain`) is

$$y(\xi) = L\,\frac{1 + \xi}{1 - \xi + 2L/y_{\max}}, \qquad L = \frac{y_{\max}}{6} \text{ (stretching parameter).} \tag{4}$$

This sends $\xi = +1$ (node j=0) to $y = y_{\max}$ (**freestream**) and $\xi = -1$ (node j=N) to $y = 0$ (**wall**); the grid median sits at $y(\xi{=}0) = y_{\max}/8$ and roughly half the points lie below $y \approx L$. By the chain rule, with $\xi_y = 1/y_\xi$ and $\xi_{yy} = -y_{\xi\xi}/y_\xi^3$, the physical operators are

$$D_1 = \operatorname{diag}(\xi_y)\, D_\xi, \qquad D_2 = \operatorname{diag}(\xi_y^2)\, D_\xi^2 + \operatorname{diag}(\xi_{yy})\, D_\xi. \tag{5}$$

Common pitfall. The chain rule is applied to the *computational* second-derivative matrix $D_\xi^2 = D_\xi D_\xi$, then mapped by (5). The physical D_2 is *not* $D_1 D_1$.

The physical domain must reach well past the boundary layer, so the truncation is set as a *multiple of the boundary-layer scale*, $y_{\max} = m\,(\delta^*/L^*)$, with $m \approx 8$–12 for the wall-trapped second mode and larger ($m \approx 35$–45) for the more diffuse first mode; the bare values $y_{\max} = 6/12$ are only a fallback. The default resolution is $N = 128$ (n=N+1=129); N is a free accuracy parameter, and because the method is spectral the eigenvalue converges *exponentially* in N (Figure 2).

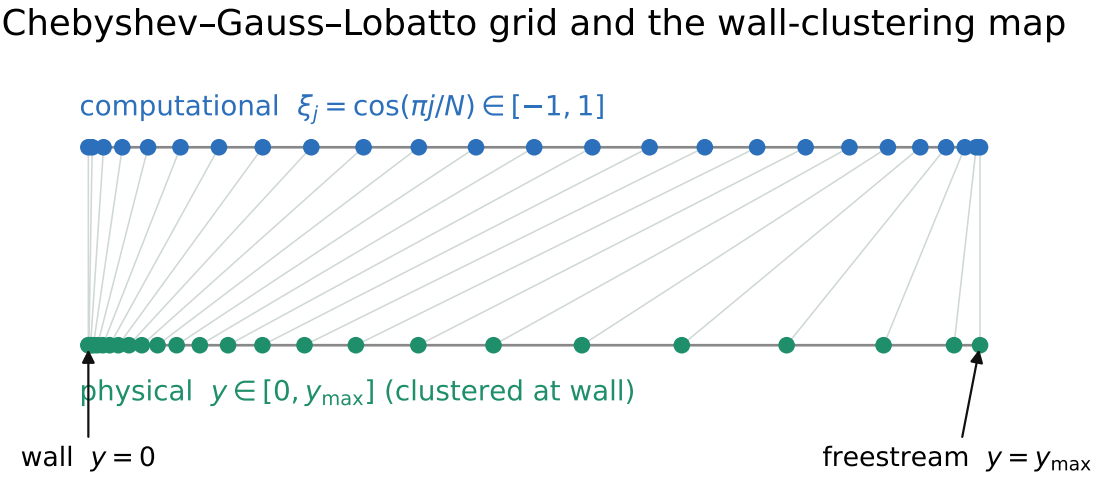

Figure 1: The GLL nodes $\xi_j = \cos(\pi j/N)$ on $[-1, 1]$ (top) map by Eq. (4) to the physical grid $y \in [0, y_{\max}]$ (bottom), piling up at the wall ($\xi = -1$) where eigenfunctions vary fastest.

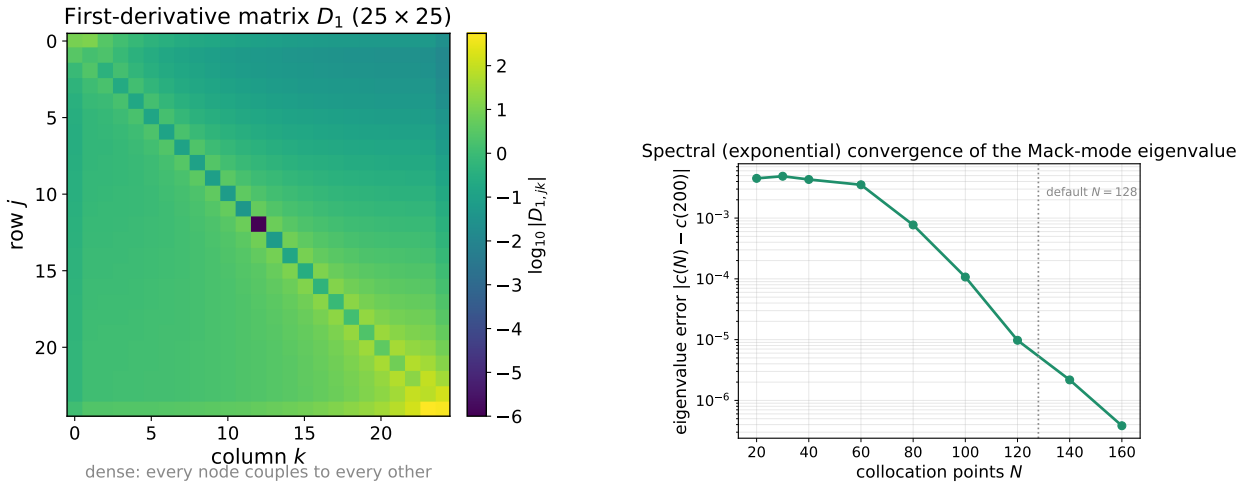

Figure 2: *Left:* the physical first-derivative matrix D_1 (25×25 here) is dense ($\log_{10}$ of the entry magnitudes shown); spectral accuracy couples every node to every other. *Right:* the payoff — the worked-example eigenvalue error $|c(N) - c(200)|$ falls *exponentially* with N (a straight line on this semilog axis), so the default $N = 128$ already gives many digits.

2 Assembling the operator and its boundary conditions

What problem is solved here. Stack the four nodal-value vectors into one length-$4n$ unknown and turn the four disturbance equations into one $4n \times 4n$ matrix (two matrices, for the generalised problem). Then overwrite the wall/freestream rows with boundary conditions.

An ODE in y is an *infinite-dimensional* statement ("this holds for every y"); a computer needs something finite. We get there in four mechanical steps that turn the four disturbance equations into the two matrices A, B. The worked example and complete block listing that follow simply carry these steps out.

Step 1 — vectorise the unknowns. Sample each amplitude at the n grid points, so the *function* $\hat{u}(y)$ is represented by the *vector* of its nodal values $\hat{u} = (\hat{u}(y_0), \dots, \hat{u}(y_{n-1}))^{\mathsf{T}}$, and stack the four fields into one column,

$$\boldsymbol{\phi} = [\hat{u}; \hat{v}; \hat{T}; \hat{p}] \in \mathbb{C}^{4n}.$$

This $\boldsymbol{\phi}$ is the discrete eigenvector; un-stacked it is the mode shape $\hat{u}(y), \hat{v}(y), \hat{T}(y), \hat{p}(y)$.

Step 2 — replace each operation by its matrix. Every operation acting on those samples has a matrix counterpart, so a continuous term turns into a matrix multiplying a field-vector:

operation in the ODE	becomes	example
$\mathrm{D} = \mathrm{d}/\mathrm{d}y$ (and D^2)	the differentiation matrix D_1 (and D_2), *dense*	$\hat{u}'(y) \to D_1 \hat{u}$
multiply by a mean profile $f(y)$	the *diagonal* $\mathrm{diag}\big(f(y_j)\big)$	$\overline{U}\,\hat{u} \to \mathrm{diag}(\overline{U})\,\hat{u}$
multiply by a scalar $\mathrm{i}\alpha$, $-\alpha^2$	that scalar times I	$\mathrm{i}\alpha\,\hat{u} \to \mathrm{i}\alpha I\,\hat{u}$

Multiplication becomes *diagonal* because $f(y)$ merely rescales each sample by its local value $f(y_j)$; differentiation couples every node to every other, so D_1, D_2 are full. A combined term keeps the order $f\,\mathrm{D} \to \mathrm{diag}(f)\,D_1$ (coefficient times derivative matrix).

Step 3 — each equation becomes a block-row. Read an equation left to right: every term drops into the block addressed by *(its equation's row, its field's column)*. Continuity, x-momentum, y-momentum and energy thus fill rows C, X, Y, E, while the fields $\hat{u}, \hat{v}, \hat{T}, \hat{p}$ index the columns. Because each equation is imposed at all n nodes it furnishes n rows, so every block is $n \times n$; four equations $\times$ four fields make a 4×4 grid of $n \times n$ blocks — a single $4n \times 4n$ matrix (Figure 3).

Step 4 — split off the eigenvalue. The eigenvalue c enters *only linearly*, through the convective factor $\mathrm{i}\alpha(\overline{U} - c)$ (and, in continuity, the state-law terms it multiplies); it never appears squared or inside a derivative. Writing $\mathrm{i}\alpha(\overline{U} - c) = \mathrm{i}\alpha\overline{U} - c\,\mathrm{i}\alpha$ and collecting the whole discretised stack by its dependence on c,

$$\underbrace{(c\text{-free terms})}_{A}\,\boldsymbol{\phi} \;-\; c\,\underbrace{(c\text{-coefficient terms})}_{B}\,\boldsymbol{\phi} = 0 \qquad \Longleftrightarrow \qquad \boxed{A\,\boldsymbol{\phi} = c\,B\,\boldsymbol{\phi}}.$$

The two matrices drop out directly: A is the c-free part — equivalently the full Step-3 block operator with every $\mathrm{i}\alpha(\overline{U} - c) \to \mathrm{i}\alpha\overline{U}$, so **its rows are exactly the four disturbance equations** — and $B = -\partial(\text{operator})/\partial c$. The problem is *generalised* (a second matrix B on the right, not

merely $c\boldsymbol{\phi}$) precisely because c multiplies only those few terms: A is block-dense, while B holds only five non-zero blocks,

$$B_{(C,\hat{T})} = -\frac{\mathrm{i}\alpha}{\overline{T}}, \quad B_{(C,\hat{p})} = \mathrm{i}\alpha\gamma M_e^2, \quad B_{(X,\hat{u})} = \mathrm{i}\alpha, \quad B_{(Y,\hat{v})} = \mathrm{i}\alpha, \quad B_{(E,\hat{T})} = \mathrm{i}\alpha. \tag{6}$$

Worked example — the continuity block-row, built term by term. Apply the recipe to the continuity equation, splitting the convective factor $\mathrm{i}\alpha(\overline{U}-c) = \mathrm{i}\alpha\overline{U} - c\,\mathrm{i}\alpha$ so the $\mathrm{i}\alpha\overline{U}$ part stays in A and the c-coefficient $\mathrm{i}\alpha$ goes to B. Reading off the four fields:

$$\begin{aligned} A_{(C,\hat{u})} &= \mathrm{i}\alpha I, & A_{(C,\hat{v})} &= D_1 - \mathrm{diag}(\overline{T}'/\overline{T}), \\ A_{(C,\hat{p})} &= \mathrm{i}\alpha\gamma M_e^2\,\mathrm{diag}(\overline{U}), & A_{(C,\hat{T})} &= -\mathrm{i}\alpha\,\mathrm{diag}(\overline{U}/\overline{T}), \\ B_{(C,\hat{p})} &= \mathrm{i}\alpha\gamma M_e^2 I, & B_{(C,\hat{T})} &= -\mathrm{i}\alpha\,\mathrm{diag}(1/\overline{T}). \end{aligned}$$

Six non-zero entries — four in A, two in B. The eigenvalue here multiplies the state-law $\hat{p}, \hat{T}$ terms (not a convective velocity), which is why row C contributes *two* B-blocks while the momentum/energy rows each contribute one. Every other equation is processed identically.

All sixteen A-blocks (complete listing, transcribed from `pymack/temporal_solver.py`). Rows are the four equations, columns the four fields $(\hat{u}, \hat{v}, \hat{T}, \hat{p})$; $\nu_R = 1/(Re\overline{\rho})$, with conduction/dissipation prefactors $\mathcal{C}_\kappa = \gamma\overline{\mu}/(Pr\,Re\,\overline{\rho})$, $\mathcal{C}_d = \gamma(\gamma-1)M_e^2/(Re\,\overline{\rho})$ and conduction operator $\mathcal{L}_c = \mathrm{D}^2 - \alpha^2 + \frac{1}{\overline{\kappa}}\frac{\mathrm{d}\kappa}{\mathrm{d}T}(2\overline{T}'\mathrm{D} + \overline{T}'') + \frac{1}{\overline{\kappa}}\frac{\mathrm{d}^2\kappa}{\mathrm{d}T^2}(\overline{T}')^2$, and a bare mean symbol denotes its diagonal matrix. The five B-blocks are Eq. (6); *every block not listed is zero* — notably $A_{(E,\hat{p})} = 0$, since the energy row has no pressure term.

(C) $\hat{u} \to \mathrm{i}\alpha I$; $\hat{v} \to D_1 - \overline{T}'/\overline{T}$; $\hat{T} \to -\mathrm{i}\alpha\overline{U}/\overline{T}$; $\hat{p} \to \mathrm{i}\alpha\gamma M_e^2\overline{U}$.

(X) $\hat{u} \to \mathrm{i}\alpha\overline{U} - \nu_R(\overline{\mu}D_2 + \overline{\mu}'D_1) + \frac{4}{3}\alpha^2\nu_R\overline{\mu}$; $\hat{v} \to \overline{U}' - \mathrm{i}\alpha\nu_R(\frac{1}{3}\overline{\mu}D_1 + \overline{\mu}')$; $\hat{T} \to -\nu_R\big[\frac{\mathrm{d}\mu}{\mathrm{d}T}(\overline{U}'D_1 + \overline{U}'') + \frac{\mathrm{d}^2\mu}{\mathrm{d}T^2}\overline{U}'\overline{T}'\big]$; $\hat{p} \to \mathrm{i}\alpha\overline{\rho}^{-1}$.

(Y) $\hat{u} \to -\mathrm{i}\alpha\nu_R(\frac{1}{3}\overline{\mu}D_1 - \frac{2}{3}\overline{\mu}')$; $\hat{v} \to \mathrm{i}\alpha\overline{U} - \nu_R(\frac{4}{3}\overline{\mu}D_2 + \frac{4}{3}\overline{\mu}'D_1) + \alpha^2\nu_R\overline{\mu}$; $\hat{T} \to -\mathrm{i}\alpha\nu_R\frac{\mathrm{d}\mu}{\mathrm{d}T}\overline{U}'$; $\hat{p} \to \overline{\rho}^{-1}D_1$.

(E) $\hat{u} \to \mathrm{i}\alpha(\gamma-1)\overline{\rho}^{-1} - 2\mathcal{C}_d\overline{\mu}\overline{U}'D_1$; $\hat{v} \to \overline{T}' + (\gamma-1)\overline{\rho}^{-1}D_1 - 2\mathrm{i}\alpha\mathcal{C}_d\overline{\mu}\overline{U}'$; $\hat{T} \to \mathrm{i}\alpha\overline{U} - \mathcal{C}_\kappa\mathcal{L}_c - \mathcal{C}_d\frac{\mathrm{d}\mu}{\mathrm{d}T}(\overline{U}')^2$; $\hat{p} \to 0$.

Holding these against `temporal_solver.py` line for line: row C is `A[blk(0,·)]`, row X `A[blk(1,·)]`, and so on; the diagonals $\overline{U}, \overline{\mu}, \ldots$ are the `np.diag(...)` arrays, D_1, D_2 the differentiation matrices, and ν_R = `rhob_inv/Re`. Figure 4 shows what this listing looks like once actually assembled into numbers, for a small solved case.

Boundary conditions = row replacement, with explicit indices. A collocation row says "the equation holds at node y_j." At the wall and freestream we overwrite those rows. With the stacking $\boldsymbol{\phi} = [\hat{u}; \hat{v}; \hat{T}; \hat{p}]$, field block $b \in \{0,1,2,3\}$ occupies global indices $bn, \ldots, bn+n-1$, and (from the ordering above) local index 0 is the freestream, local index $n-1$ the wall. Hence:

condition	field	global row	row becomes
no-slip (wall)	$\hat{u}$	$n-1$	identity ($\hat{u}=0$)
no-slip (wall)	$\hat{v}$	$2n-1$	identity ($\hat{v}=0$)
thermal (wall)	$\hat{T}$	$3n-1$	identity ($\hat{T}=0$, isothermal) *or* D_1[wall, :] ($\mathrm{D}\hat{T}=0$, adiabatic)
decay (freestream)	$\hat{u}$	0	identity
decay (freestream)	$\hat{v}$	n	identity
decay (freestream)	$\hat{T}$	$2n$	identity

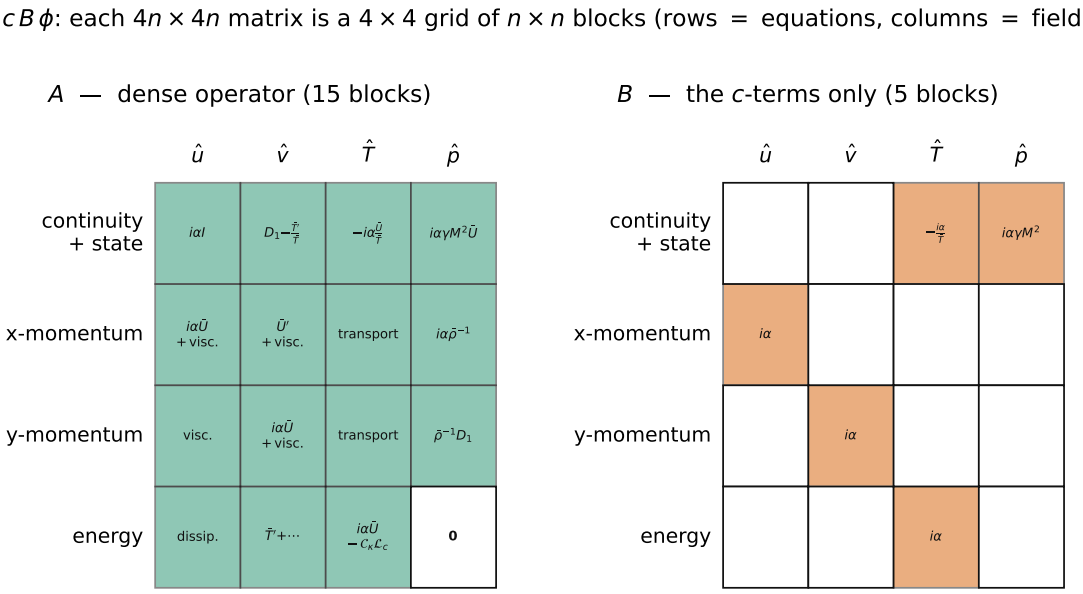

Figure 3: Block structure of $A\boldsymbol{\phi} = cB\boldsymbol{\phi}$. Rows = the four equations (continuity, x-mom, y-mom, energy); columns = the four fields $\hat{u}, \hat{v}, \hat{T}, \hat{p}$. *Shaded = non-zero block.* A (left) is block-dense; B (right) holds only the five blocks of Eq. (6) — every term carrying the eigenvalue c.

The same rows of B are zeroed so the constraint is algebraic; $\hat{p}$ carries no explicit boundary row.

Getting this backwards silently fails. Because $\xi_0 = +1 \to y_{\max}$ and $\xi_N = -1 \to y = 0$, node index 0 is the freestream and index $n-1$ is the wall — the opposite of the naive guess. Swapping them produces plausible-looking but wrong eigenvalues.

3 Temporal problem: a generalised eigenvalue problem

What problem is solved here. **Given** a real wavenumber α, **find** the complex phase speed c and shape $\boldsymbol{\phi}$. **Equation:** the four disturbance equations. **Eigenvalue:** c. **Growth:** $\omega_i = \alpha\,\mathrm{Im}(c)$.

With A, B from §2 this is a **generalised eigenvalue problem**

$$A\boldsymbol{\phi} = cB\boldsymbol{\phi}, \qquad A, B \in \mathbb{C}^{4n\times 4n}, \tag{7}$$

solved directly by the QZ algorithm (`scipy.linalg.eig`), which returns all $4n$ eigenvalues at once.

In plain words. *An ordinary eigenproblem is $A\phi = c\phi$; a* generalised *one allows a second matrix B on the right. The discrete eigenvector $\boldsymbol{\phi}$ is just the eigenfunction $\hat{q}(y)$ sampled at the grid points — "eigenfunction" and "eigenvector" are the continuous and discrete names for the same shape.*

A disturbance $\propto e^{\mathrm{i}\alpha(x-ct)}$ with α real behaves as $e^{\alpha c_i t}$ in time, so

$$c_i = \mathrm{Im}(c) > 0 \;\Rightarrow\; \textbf{unstable (grows in time)}, \qquad c_r = \mathrm{Re}(c) = \text{wave speed}. \tag{8}$$

The wave speed classifies the mode: a first (viscous/Tollmien–Schlichting) mode has $c_r \approx 0.3$–0.6; a second (Mack) mode has $c_r \approx 1 - 1/M_e$. The raw spectrum is filtered to $\mathrm{Re}(c) \in (-0.5, 1.5)$, $|\,\mathrm{Im}(c)| < 0.5$ and sorted by c_i.

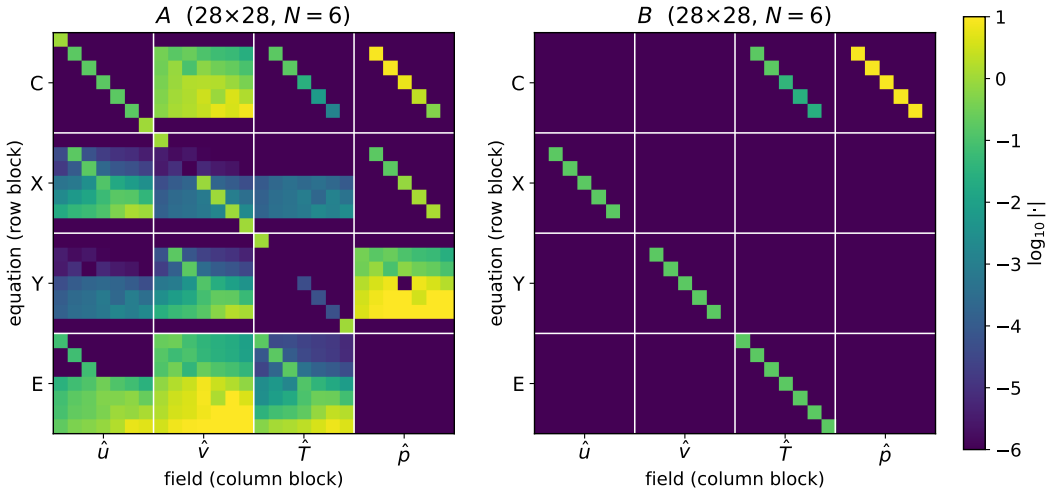

Figure 4: The same block structure, but numerically real: A and B assembled from an actual solved case (a small worked example, $N = 6$ so $4n = 28$, at the §6 parameters), shown as $\log_{10}|\cdot|$ so both magnitude and sparsity are visible. Grey gridlines every n rows/columns mark the same C, X, Y, E row blocks and $\hat{u}, \hat{v}, \hat{T}, \hat{p}$ column blocks labelled symbolically in Figure 3: A is densely populated throughout, while B lights up only its five algebraic blocks, exactly as predicted.

4 Spatial problem: a quadratic eigenvalue problem

What problem is solved here. **Given** a real frequency ω, **find** the complex wavenumber α and shape. **Equation:** the same physics grouped by powers of α. **Eigenvalue:** α. **Growth:** $\sigma = -\operatorname{Im}(\alpha)$.

A real source forces a fixed frequency and the disturbance grows in *space*. Now ω is real and α is the unknown; because viscous/conduction terms carry $\partial_x^2 \mapsto -\alpha^2$, α appears *quadratically*. Grouping the discretised disturbance equations by powers of α (substitute $\mathrm{D} \to D_1$, $\mathrm{D}^2 \to D_2$, coefficients $\to$ diagonals, then collect $\alpha^0, \alpha^1, \alpha^2$) gives a **quadratic eigenvalue problem** (QEP)

$$\big(C_0 + \alpha C_1 + \alpha^2 C_2\big)\boldsymbol{\phi} = 0. \tag{9}$$

Opening up C_0, C_1, C_2 (complete listing). The spatial assembler (`equations.py`) uses the *enthalpy* energy form — so $\hat{p}$ appears explicitly via $\mathrm{D}p'/\mathrm{D}t$ and the temperature equation differs from the temporal energy row — and a bulk-viscosity ratio $\lambda/\mu = 1.2$. The resulting viscous coefficients are

$$c_{\alpha^2} = \tfrac{2(2+\lambda/\mu)}{3} = 2.13, \qquad c_\times = \tfrac{1+2\lambda/\mu}{3} = 1.13, \qquad c_a = \tfrac{2(\lambda/\mu-1)}{3} = 0.13 \quad \text{(Mack Eq. 8.5)}. \tag{10}$$

Because this closure is *not* obtained by re-grouping the temporal equations, here are *all* non-zero blocks, transcribed from `equations.py`. Write $\nu_R = 1/(Re\overline{\rho})$ for the viscous/conduction prefactor and $d_R = (\gamma-1)M_e^2\nu_R$ for the dissipation prefactor; each entry below is the operator acting on the named field in that equation's block-row.

(C) C_0: $\hat{v} \to D_1 - \frac{\overline{T}'}{\overline{T}}$, $\hat{T} \to \frac{\mathrm{i}\omega}{\overline{T}}$, $\hat{p} \to -\mathrm{i}\omega\gamma M_e^2$. C_1: $\hat{u} \to \mathrm{i}I$, $\hat{T} \to -\frac{\mathrm{i}\overline{U}}{\overline{T}}$, $\hat{p} \to \mathrm{i}\gamma M_e^2 \overline{U}$. C_2: none.

(X) C_0: $\hat{u} \to -\mathrm{i}\omega - \nu_R(\overline{\mu}D_2 + \overline{\mu}' D_1)$, $\hat{v} \to \overline{U}'$, $\hat{T} \to -\nu_R\big[\frac{\mathrm{d}\mu}{\mathrm{d}T}(\overline{U}'' + \overline{U}' D_1) + \frac{\mathrm{d}^2\mu}{\mathrm{d}T^2}\overline{U}'\overline{T}'\big]$. C_1: $\hat{u} \to \mathrm{i}\overline{U}$, $\hat{v} \to -\mathrm{i}\nu_R(c_\times \overline{\mu} D_1 + \overline{\mu}')$, $\hat{p} \to \mathrm{i}\overline{\rho}^{-1}$. C_2: $\hat{u} \to c_{\alpha^2}\nu_R\overline{\mu}$.

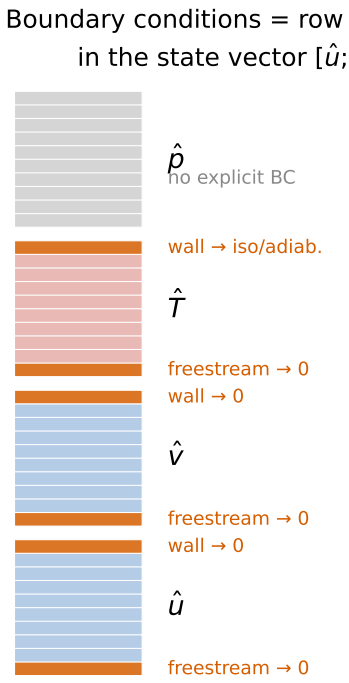

Figure 5: The boundary conditions as row replacement: the stacked state $[\hat{u};\hat{v};\hat{T};\hat{p}]$ with the six overwritten rows highlighted (wall and freestream for $\hat{u},\hat{v},\hat{T}$); $\hat{p}$ carries no boundary row. Note index 0 is the freestream and $n-1$ the wall.

(Y) C_0: $\hat{v}\to -\mathrm{i}\omega - c_{\alpha^2}\nu_R(\overline{\mu}D_2+\overline{\mu}'D_1)$, $\hat{p}\to \overline{\rho}^{-1}D_1$. C_1: $\hat{u}\to -\mathrm{i}\nu_R(c_\times\overline{\mu}D_1+c_a\overline{\mu}')$, $\hat{v}\to \mathrm{i}\overline{U}$, $\hat{T}\to -\mathrm{i}\nu_R\frac{\mathrm{d}\mu}{\mathrm{d}T}\overline{U}'$. C_2: $\hat{v}\to \nu_R\overline{\mu}$.

(E) C_0: $\hat{u}\to -2d_R\overline{\mu}\overline{U}'D_1$, $\hat{v}\to \overline{T}'$, $\hat{T}\to -\mathrm{i}\omega-\nu_R\big[\overline{\kappa}D_2+2\overline{\kappa}'D_1+\frac{\mathrm{d}\kappa}{\mathrm{d}T}\overline{T}''+\frac{\mathrm{d}^2\kappa}{\mathrm{d}T^2}(\overline{T}')^2\big]-d_R\frac{\mathrm{d}\mu}{\mathrm{d}T}(\overline{U}')^2$, $\hat{p}\to \mathrm{i}\omega(\gamma-1)M_e^2\overline{\rho}^{-1}$. C_1: $\hat{v}\to -2\mathrm{i}d_R\overline{\mu}\overline{U}'$, $\hat{T}\to \mathrm{i}\overline{U}$, $\hat{p}\to -\mathrm{i}(\gamma-1)M_e^2\overline{U}\overline{\rho}^{-1}$. C_2: $\hat{T}\to\nu_R\overline{\kappa}$.

All other blocks are zero. The temporal closure (Stokes $\frac{4}{3},\frac{1}{3}$ factors, temperature energy row) and this spatial closure (enthalpy row, $\lambda/\mu=1.2$) differ only in the bulk-viscosity treatment and the energy-equation arrangement; both approximate the same physical mode and agree on it to within discretisation error.

Companion linearisation. A QEP becomes an ordinary (generalised) problem of *twice* the size by introducing $\boldsymbol{\psi}=\alpha\boldsymbol{\phi}$ (`solver.py`):

$$\underbrace{\begin{bmatrix}-C_1 & -C_0\\ I & 0\end{bmatrix}}_{\mathcal{L}}\begin{bmatrix}\boldsymbol{\psi}\\ \boldsymbol{\phi}\end{bmatrix}=\alpha\underbrace{\begin{bmatrix}C_2 & 0\\ 0 & I\end{bmatrix}}_{\mathcal{R}}\begin{bmatrix}\boldsymbol{\psi}\\ \boldsymbol{\phi}\end{bmatrix}. \tag{11}$$

The lower block row enforces $\boldsymbol{\psi}=\alpha\boldsymbol{\phi}$; the upper row then reproduces (9). The eigenvalue is α and the physical shape is the *lower* half $\boldsymbol{\phi}$ (Figure 7).

Why "the lower half". Since $\boldsymbol{\psi}=\alpha\boldsymbol{\phi}$ the two halves are parallel, so both satisfy the QEP up to scaling; by convention $\boldsymbol{\phi}$ (lower half) is the one normalised as the eigenfunction.

Shift–invert (which eigenvalue, cheaply). We target the relevant branch with a shift $\alpha_0=\omega/c_{\mathrm{ph}}$ ($c_{\mathrm{ph}}=1-1/M_e$ for $M_e>3$, else ≈ 0.4). Subtracting α_0 and inverting maps eigenvalues

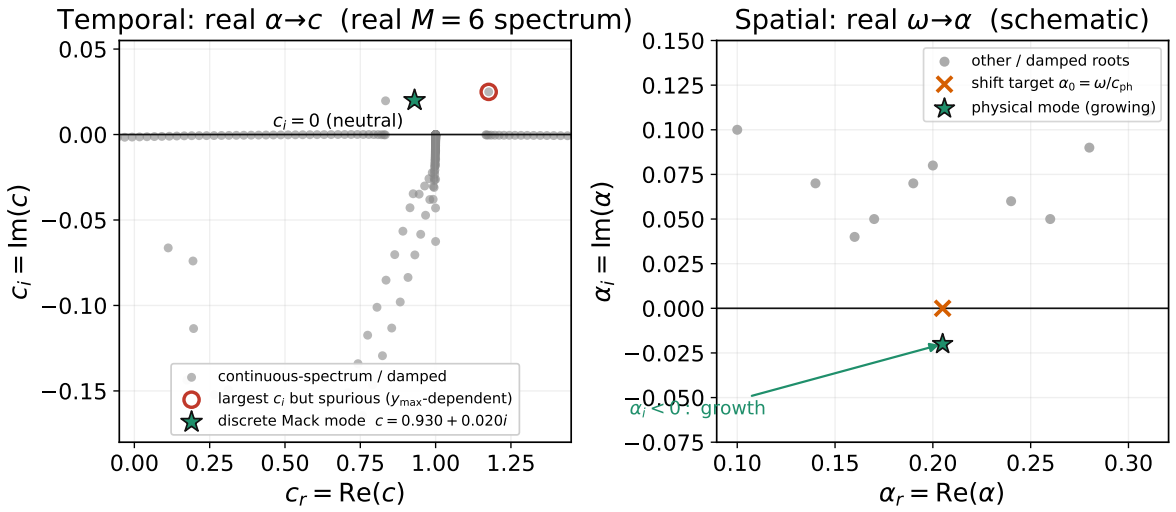

Figure 6: Where the eigenvalues live. *Left (temporal):* input real α; the phase speeds c populate the complex plane; the physical discrete mode sits above the line $c_i = \mathrm{Im}(c) = 0$ (unstable) while the continuous spectrum is damped. *Right (spatial, explained in §4):* input real ω; the wavenumbers α are sought near a shift target α_0; the physical mode has $\alpha_i < 0$, i.e. growth.

near α_0 to *large* μ, where

$$\text{eigenvalues of } (\mathcal{L} - \alpha_0\mathcal{R})^{-1}\mathcal{R} \;\text{ are }\; \mu = \frac{1}{\alpha - \alpha_0}, \qquad \Rightarrow \qquad \alpha = \alpha_0 + \frac{1}{\mu}. \tag{12}$$

So returning the largest-$|\mu|$ modes returns the wavenumbers nearest the physical one (one LU factorisation; in `pyMack` a dense solve followed by selecting the n_{modes}=20 nearest α_0, then sorting by $\mathrm{Im}(\alpha)$). The amplitude $\propto e^{\mathrm{i}\alpha x} = e^{\mathrm{i}\alpha_r x} e^{-\alpha_i x}$, so

$$\sigma = -\,\mathrm{Im}(\alpha) = -\alpha_i > 0 \;\Rightarrow\; \textbf{unstable (grows downstream).} \tag{13}$$

Alternative route — Newton with a phase-speed seed. A more robust path when shift–invert struggles (`solve_spatial_from_temporal`): solve the *temporal* problem at a real trial $\alpha_r = \omega/c_{\text{est}}$ ($c_{\text{est}} = 0.98$ for $M_e > 3$, else 0.35), keeping only modes that agree across two resolutions (tolerance 8×10^{-3}); form the seed $\mathrm{Im}(\alpha) \approx -\alpha_r c_i / c_r$ (a *phase-speed* estimate — the rigorous Gaster relation uses the group velocity $c_g = \mathrm{d}\omega_r/\mathrm{d}\alpha_r$, but here it only seeds Newton, so precision is unneeded); then Newton-iterate the dispersion residual $F(\alpha) = \alpha c(\alpha) - \omega = 0$, $\alpha^{(k+1)} = \alpha^{(k)} - F/F'$, with $F' = c + \alpha\,\mathrm{d}c/\mathrm{d}\alpha$ approximated by a finite difference (step 10^{-5}, tol 10^{-6}, ≤ 20 iterations). Both routes return the same $\sigma = -\,\mathrm{Im}(\alpha)$.

5 Which eigenvalue is physical?

What problem is solved here. Given the full $4n$-element spectrum, identify the one eigenvalue that is a genuine boundary-layer mode rather than continuous-spectrum noise.

Besides the discrete boundary-layer modes the operator has a dense **continuous spectrum** — on an infinite domain these freestream disturbances form an unbroken band rather than isolated points, and the finite grid renders them as a dense cluster. Two tests (`discrete_mode.py`) keep only the physical mode (Figure 8):

1. **Freestream decay**: a true boundary-layer eigenfunction decays as $y \to y_{\max}$, a continuous-spectrum one does not. Reject candidates whose edge-to-peak amplitude ratio exceeds $\sim$0.06.

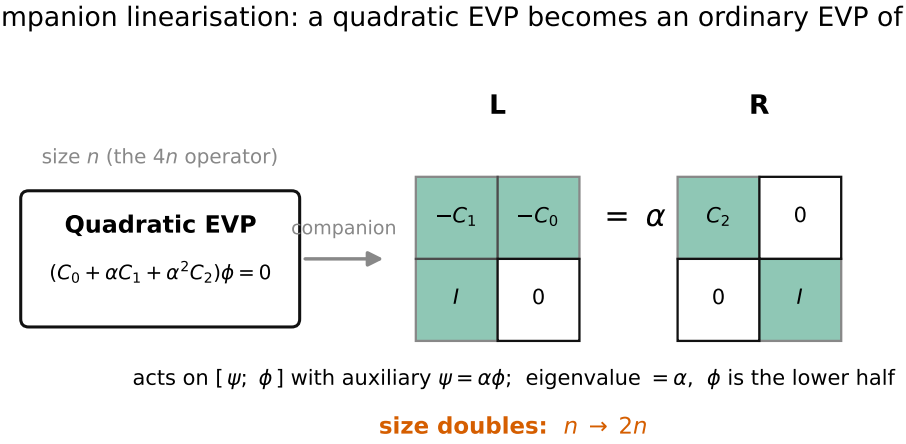

Figure 7: Companion linearisation (11): the $4n$-sized quadratic problem becomes an ordinary generalised problem of size $8n$ in $[\psi; \phi]$, with $\psi = \alpha\phi$. Its eigenvalue is exactly α; the eigenfunction is the lower half.

2. **Domain stationarity**: a genuine eigenvalue barely moves if the truncation changes. Recompute at two domain heights and keep modes whose c matches to $\sim 4 \times 10^{-3}$; accept growth rates with $|c_i| \lesssim 0.05$ and c_r in the physical band.

In plain words. *These two tests are the difference between a clean growth rate and noise — they are what let us trust a single number out of the full $4n$-element spectrum (724 modes in the worked example of §6, where the largest-growth eigenvalue is in fact spurious).*

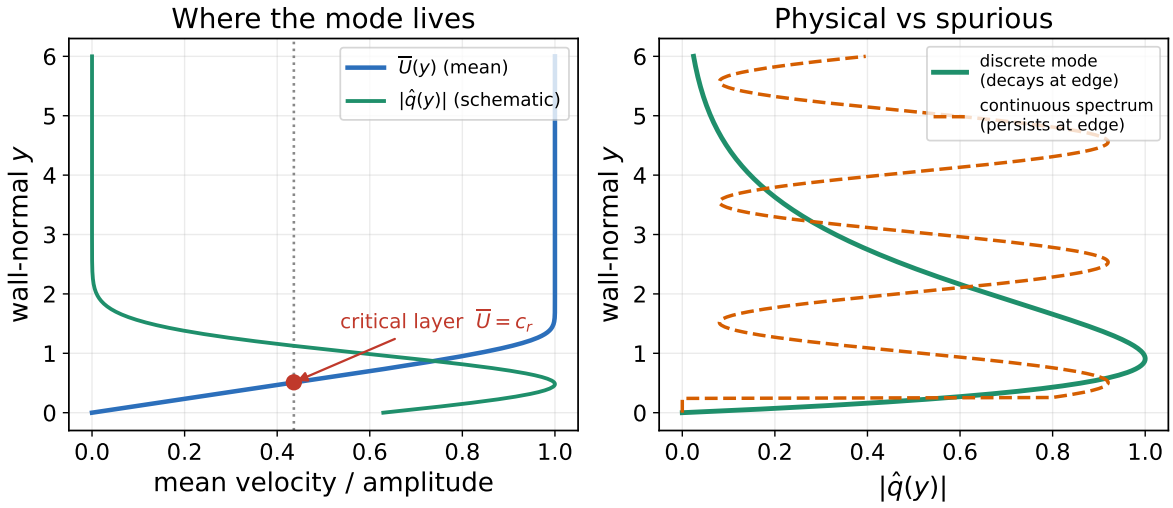

Figure 8: *Left:* the critical layer is where $\overline{U}(y) = c_r$; the mode's amplitude $|\hat{q}(y)|$ tends to peak near it. *Right:* a discrete boundary-layer mode (solid) decays toward the freestream, while a continuous-spectrum mode (dashed) keeps oscillating to the edge — the basis for the two acceptance tests.

6 Worked example: one temporal solve, start to finish

Take a strong Mack-mode case: $M_e = 6.0$, $R = 5500$, $\alpha_L = 0.174$, adiabatic wall, L^* scaling, $N = 180$ ($n = 181$, so the matrices are $4n = 724$ wide). The wall-normal domain is $y_{\max} = 10\,(\delta^*/L^*) \approx 128$ (§1: the domain is a multiple of the boundary-layer scale, not a bare number). Solving (7) returns 724 eigenvalues; after the physical filter the six least-stable are

c_r	c_i	$\omega_i = \alpha c_i$	$y_{\max}$-stationary?	verdict
+1.176	+0.0249	+0.00434	no ($\lvert\Delta c\rvert \sim 10^{-2}$)	spurious / continuous spectrum
+0.930	**+0.0200**	**+0.00348**	**yes** ($\lvert\Delta c\rvert \sim 10^{-7}$)	**discrete Mack mode — the answer**
+0.834	+0.0197	+0.00343	no	continuous spectrum
+0.740	+0.0000	0.00000	—	neutral continuous-spectrum cluster
+0.720	+0.0000	0.00000	—	continuous spectrum
+0.760	+0.0000	0.00000	yes	stable discrete mode

This is the trap the mode tests of §5 exist to avoid: the *largest*-growth eigenvalue (row 1, $c_r = 1.18$) is *not* the answer — it is spurious, its c shifting by $\sim 10^{-2}$ when the domain height changes (and $c_r > 1$ is unphysically supersonic). The physical answer is row 2, the **Mack (second) mode** $c = 0.930 + 0.0200$i: domain-stationary to 10^{-7}, with an eigenfunction that decays at the freestream (Figure 9). Its growth rate is $\omega_i = \alpha c_i = +0.00348$ (unstable) and its phase speed $c_r = 0.93$ is the classic second-mode value near $1 - 1/M_e$.

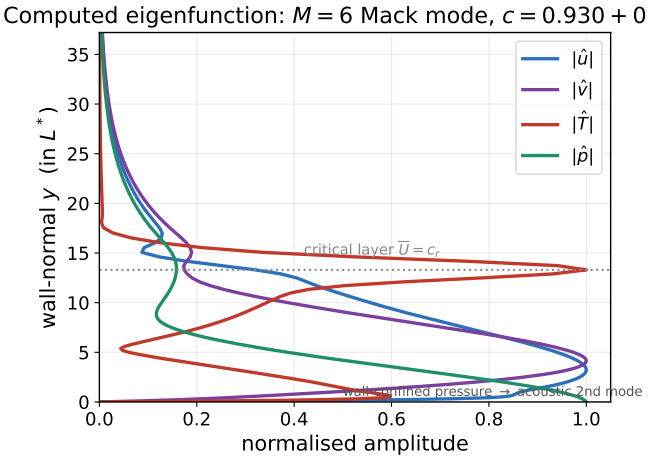

Figure 9: Computed eigenfunction of the worked-example Mack mode: the four amplitude fields $|\hat{u}|, |\hat{v}|, |\hat{T}|, |\hat{p}|$ vs y (each normalised). $|\hat{T}|$ spikes at the critical layer ($\overline{U} = c_r$); $|\hat{p}|$ is wall-confined — the acoustic signature of the second mode. All fields decay at the freestream, confirming a discrete mode.

This single $\omega_i > 0$ is one point "inside" the neutral curve: repeating the solve across an (R, α) grid and finding where ω_i crosses zero traces the neutral curve of §7. (All numbers reproduce from `solve_temporal_2d`.)

7 Neutral curves

What problem is solved here. Map the boundary between growth and decay: the locus where the growth rate is exactly zero.

The neutral curve is $c_i(R, \alpha) = 0$ (temporal) or $\sigma(R, \omega) = 0$ (spatial); inside it the flow is unstable. `pyMack` draws it in three steps:

1. **Sweep** an (R, α) grid at fixed M_e; at each node solve (7), apply the mode tests (§5), store c_i (`build_*_grid.py`).
2. **Extract** the zero contour with marching squares (a standard method for tracing a $c_i = 0$ level set through a grid of values; `extract_contours.py`); unresolved nodes are tagged stable so the loop closes.
3. **Continuation** for hard branches: at low Mach the first-mode eigenvalue sinks into the continuous spectrum. `trace_continuation.py` tracks it *by proximity* — from a clean seed it marches in R, follows the nearest c, and secant-solves $c_i = 0$ at each step (continuation = using each converged eigenvalue as the guess for the next nearby parameter value).

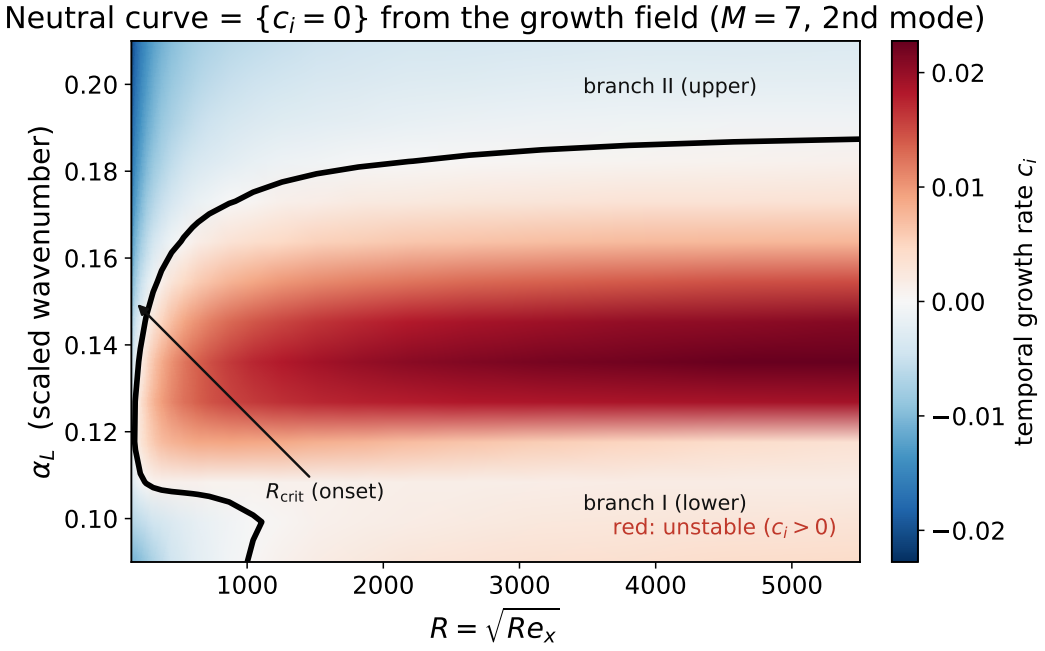

Figure 10: Step 1–2 of the neutral-curve recipe on a real case ($M_e = 7$, second mode): the coloured field is the growth rate $c_i(R, \alpha)$ from the (R, α) sweep, and the black curve is the $\{c_i = 0\}$ contour extracted from it by marching squares. The flow is unstable ($c_i > 0$) inside the loop; R_{crit} marks the lower branch (branch I) where instability first turns on as R increases, and the curve closes again at branch II.

8 N-factor and transition

What problem is solved here. Convert local spatial growth into a single number predicting transition.

What matters for transition is the *accumulated* amplification of a fixed-frequency disturbance. For a chosen reduced frequency F, solve the spatial problem (§4) at successive stations to get $\sigma_L = -\operatorname{Im}(\alpha_L)$ — this is the same spatial growth σ of Eq. (13), expressed in the L^* (Mack-length) scaling — and integrate from the lower neutral point R_0 (where σ_L first turns positive):

$$N(R) = \int_{R_0}^{R} \max(\sigma_L, 0)\, \mathrm{d}R' \quad \text{(trapezoidal)}, \qquad \frac{A(R)}{A(R_0)} = e^{N(R)}. \tag{14}$$

The amplitude grows by e^N; transition is empirically expected near $N \approx 9$–11 (the e^N method; `analysis.py`). For a cone, surface arc length is mapped to an equivalent R before integrating.

e^N is a correlation, not a criterion. The transition N is *not* universal: it depends on the disturbance environment (flight ∼9–11; noisy conventional tunnels can transition at $N \sim 4$–6). It presumes a single dominant linear mode and stops at the onset of nonlinear breakdown, and it silently absorbs the unknown initial amplitude set by receptivity.

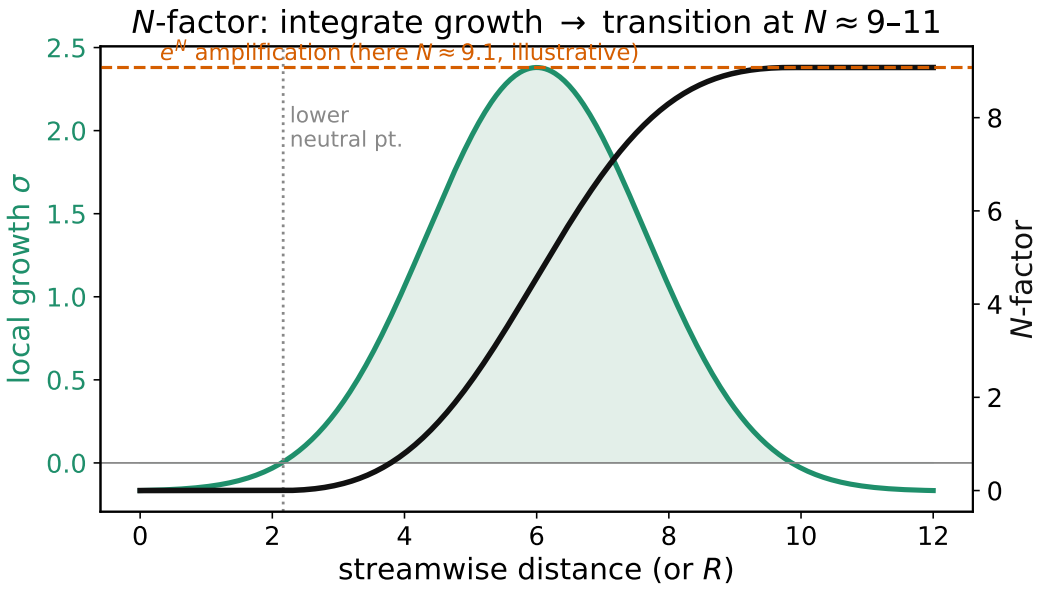

Figure 11: Illustrative N-factor integration, Eq. (14): the shaded area is $\max(\sigma_L, 0)$, the local spatial growth rate carried forward from station to station; the black curve is the running integral $N(R)$. The curve shape is schematic (a rescaled synthetic $\sigma_L(R)$), not a literal solved case, but it is drawn to reach the empirical transition band $N \approx 9$–11 so its asymptote can be read directly against that criterion.

9 Part A summary and replication checklist

Goal	Problem solved	Method	Growth
Temporal growth, neutral curves	generalised EVP $A\boldsymbol{\phi} = cB\boldsymbol{\phi}$, real α (Eq. 7)	Chebyshev collocation + QZ	αc_i
Spatial growth, N-factor	quadratic EVP $(C_0 + \alpha C_1 + \alpha^2 C_2)\boldsymbol{\phi} = 0$, real ω (Eq. 9)	companion + shift–invert; or temporal + Newton seed	$-\alpha_i$
Neutral curve	$\{c_i = 0\}$ over an (R, α) sweep (Figure 10)	grid sweep → marching squares; continuation	—
Transition	$N = \int \max(\sigma_L, 0)\, \mathrm{d}R$ (Eq. 14, Figure 11)	fixed-F spatial march + integration	—

To re-implement the spectral solver from scratch.

1. Solve the base-flow boundary-value problem (see the companion Mean Boundary Layer note); tabulate $\overline{U}, \overline{T}$ and derivatives.

2. Build ξ_j (1), D_ξ (2), map to y (4), form D_1, D_2 (5).
3. Assemble the 4×4 block operator from the four disturbance equations (temporal: A, B with the split (6); spatial: C_0, C_1, C_2).
4. Overwrite the wall/freestream rows with the boundary conditions (§2 table), respecting the node ordering.
5. Solve the eigenvalue problem (temporal (7) or spatial (11)); filter for the discrete mode (§5); read c_i or $-\alpha_i$.
6. Sweep parameters; extract $\{c_i = 0\}$ for neutral curves, or integrate $-\alpha_i$ for N-factors.

Code map (Part A). spectral operators: `pymack/spectral.py`; temporal assembly: `pymack/temporal_solver.py`; spatial assembly: `pymack/equations.py`; solvers / companion / Newton: `pymack/solver.py`; mode selection: `discrete_mode.py`. Neutral-curve drivers and continuation: `build_*_grid.py`, `extract_contours.py`, and `trace_continuation.py`.

Part B — The shooting solver

Part A built one large matrix *and asked a dense eigensolver for* all *its eigenvalues. Part B solves the identical temporal problem the opposite way: recast the disturbance equations as a small first-order system, integrate it across the layer, and drive a tiny wall matrix singular to pinpoint* one *eigenvalue. This is the classical shooting method (Mack 1984), re-implemented in the file* ***`pymack/temporal_shooting.py`*** *following Ozgen & Kircali (2008).*

10 The first-order state and its coefficient matrix

What problem is solved here. **Given** a real wavenumber α and a trial phase speed c, cast the four disturbance equations as a single first-order ODE $\mathrm{D}Y = A(y)\,Y$ in a 6-component complex state. **Eigenvalue:** c (found in §14). No global matrix is ever formed.

The spectral route discretised the four second-order disturbance ODEs *all at once* on a grid. Shooting instead reduces them to a first-order system in y and integrates it like an initial-value problem. Following Ozgen & Kircali (2008), Eqs. 22–28, specialised to two dimensions (spanwise wavenumber $\beta = 0$, no spanwise velocity), the disturbance is described by the six-component complex state

$$Y(y) = \big(X_1, X_2, X_3, X_4, X_5, X_6\big)^{\mathsf{T}} \in \mathbb{C}^6, \tag{15}$$

in which X_1, X_3, X_5 are the *primary flow amplitudes*, while X_2, X_6 are the wall-normal derivatives of X_1, X_5 and X_4 is the pressure amplitude. Concretely X_1 is the streamwise-velocity amplitude, X_3 the wall-normal-velocity amplitude, and X_5 the temperature amplitude (with $X_2 = \mathrm{D}X_1$ and $X_6 = \mathrm{D}X_5$ their wall-normal derivatives; X_4 is the pressure amplitude, coupled through the continuity and y-momentum rows rather than as a plain derivative). The state thus stacks (u, u', v, p, T, T') in that order (Ozgen's convention); the two second-order momentum/energy equations and the first-order continuity relation together close the system. The whole set is written as one first-order ODE,

$$\frac{\mathrm{d}Y}{\mathrm{d}y} = A\big(y;\ \alpha, c, Re, M_e\big)\,Y, \qquad A \in \mathbb{C}^{6\times 6}, \tag{16}$$

where A is assembled *pointwise* — one 6×6 matrix at each y — from the local mean-flow data: the velocity and temperature with their derivatives $\overline{U}, \overline{U}', \overline{U}''$ and $\overline{T}, \overline{T}'$; the viscosity $\overline{\mu}$ with $\mathrm{d}\mu/\mathrm{d}T$ and

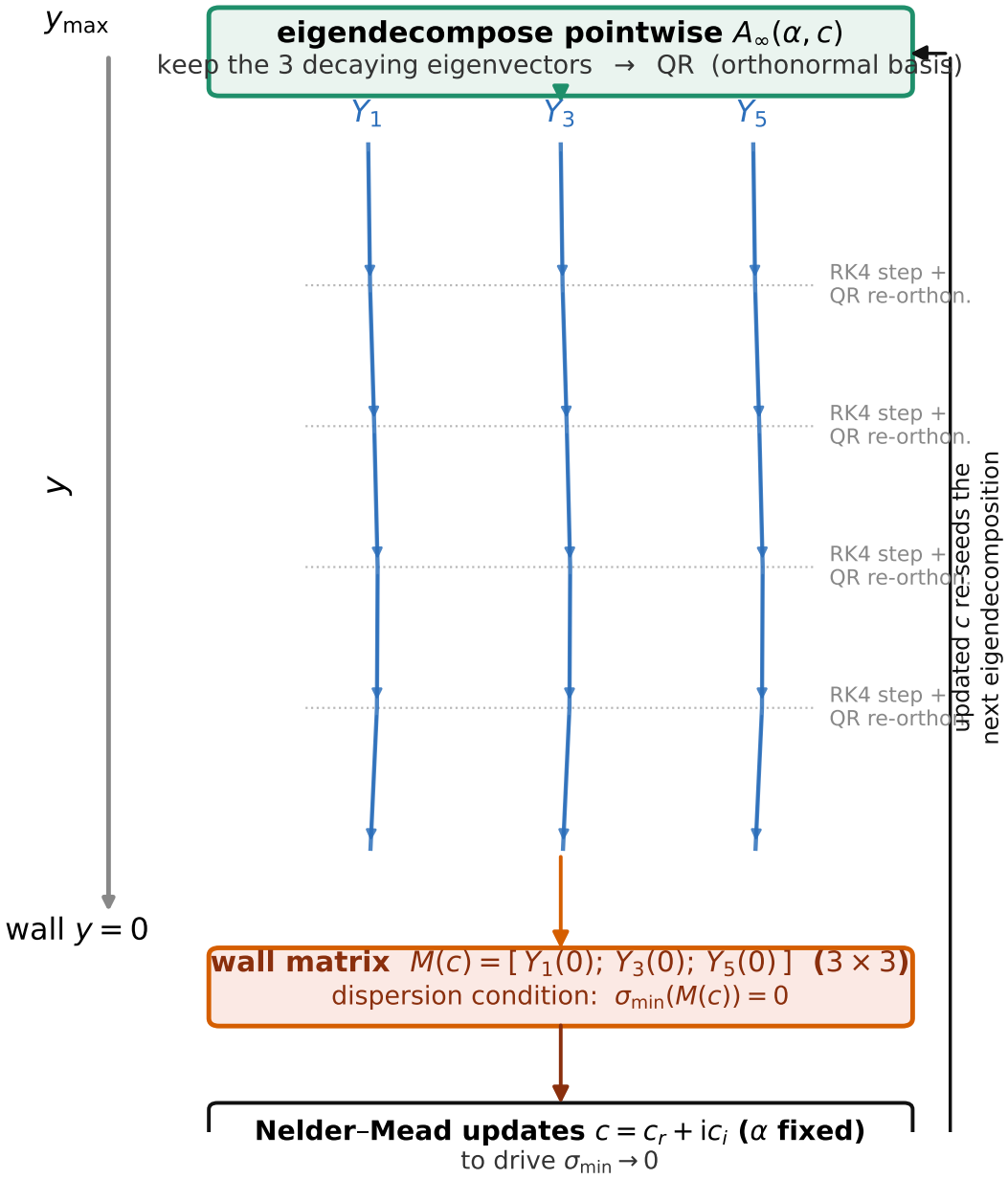

Figure 12: The whole shooting pipeline in one picture, top (y_{max}) to bottom (wall $y = 0$), matching the physical-domain sense of Figure 2: at y_{max}, A_∞ is eigen-decomposed and its three decaying eigenvectors are QR-orthonormalised into the starting 6×3 basis (§11); the basis is then RK4-marched down to the wall with a QR re-orthonormalisation after every step (§12); at the wall the three arriving columns form the 3×3 matrix $M(c)$ (§13); and an outer Nelder–Mead loop (§14) updates the trial c to drive $\sigma_{min}(c) \to 0$, feeding back into the eigen-decomposition at y_{max}.

$\mathrm{d}^2\mu/\mathrm{d}T^2$; and the conductivity $\overline{\kappa}$ with its temperature derivatives (function `ozgen_first_order_matrix_2d`, which samples the scaled base flow at the single point y).

In plain words. *Rows* 1 *and* 5 *of* A *carry the trivial "derivative-of" definitions* $\mathrm{D}X_1 = X_2$ *and* $\mathrm{D}X_5 = X_6$ *— literally the single entries* $A_{12} = 1$ *and* $A_{56} = 1$. *The physics lives in rows* $2, 4, 6$ *(the two momentum balances and the energy balance) and in row* 3 *(continuity plus the equation of state).*

The pointwise structure mirrors the block operator of Part A term for term: the convective factor $\alpha(\overline{U} - c)$, the viscous prefactor $\propto Re/(\overline{\mu}\overline{T})$, the transport-derivative couplings $\frac{\mathrm{d}\mu}{\mathrm{d}T}\overline{U}'$, and the conduction group in $\overline{\kappa}$ all reappear, only now as *scalar* entries of a 6×6 matrix evaluated at one height rather than as $n \times n$ diagonal blocks. The gas defaults ($\gamma = 1.4$) and the length scaling (`L_star`) match the spectral solver, so both routes see the identical mean flow.

11 Boundary conditions at the wall and the decaying freestream basis

What problem is solved here. Supply the six conditions that pin down a non-trivial disturbance: three homogeneous conditions at the wall, and three at the freestream selecting only the *decaying* solutions. Together they turn (16) into a well-posed shooting problem.

Wall conditions. At the wall $y = 0$ the disturbance must obey three homogeneous conditions,

$$X_1(0) = 0 \text{ (no slip)}, \qquad X_3(0) = 0 \text{ (no penetration)}, \qquad X_5(0) = 0 \text{ (isothermal wall)}, \tag{17}$$

i.e. the three *primary* flow amplitudes (u, v, T) vanish at the wall. These are exactly the collocation identity rows of the spectral solver (§2), now imposed as pointwise conditions on the integrated state.

Freestream: the decaying basis. As $y \to y_{\max}$ the mean flow becomes uniform, so the coefficient matrix $A(y_{\max})$ is essentially *constant*. A constant-coefficient linear system $\mathrm{D}Y = A_\infty Y$ has solutions $Y \propto e^{\lambda y} v$ built from the eigenpairs (λ, v) of A_∞; the six eigenvalues split into three with $\mathrm{Re}(\lambda) > 0$ (which *grow* outward and are unphysical) and three with $\mathrm{Re}(\lambda) < 0$ (which *decay* outward, the boundedness a genuine mode requires). We therefore keep only the three decaying eigenvectors (function `ozgen_freestream_decay_basis_2d`),

$$A_\infty = A(y_{\max}), \qquad A_\infty v_k = \lambda_k v_k, \qquad \mathrm{Re}(\lambda_k) < 0, \ k = 1, 2, 3, \tag{18}$$

and assemble them into a 6×3 matrix. This matrix is then orthonormalised (a QR factorisation) into an orthonormal 6×3 basis $Y(y_{\max})$ — the starting condition for the march. Enforcing decay at the edge in this way is the shooting analogue of the spectral solver's freestream identity rows.

Fallback if fewer than three decay. If a trial c makes fewer than three eigenvalues strictly decaying, the code instead takes the three *most* decaying (smallest $\mathrm{Re}\,\lambda$) so the march is always well defined; near the physical eigenvalue exactly three decay, as expected.

12 Integrating the bounded basis to the wall

What problem is solved here. Propagate the three admissible (decaying) freestream solutions down to the wall, keeping the integration numerically clean, so their wall values can be tested against (17).

Because we track three independent solutions simultaneously, the state is not a vector but the 6×3 matrix Y whose columns are the three solutions. It is marched from $y_{\max}$ *down* to the wall $y = 0$ on a fixed grid of about $n_{\text{steps}} = 800$ steps with the classical fourth-order Runge–Kutta scheme (function `integrate_ozgen_bounded_basis_2d`). Over one step of size h (here $h < 0$, since we integrate toward the wall),

$$\begin{aligned} k_1 &= A(y_0)\,Y, & k_2 &= A(y_m)\big(Y + \tfrac{1}{2}h\,k_1\big), \\ k_3 &= A(y_m)\big(Y + \tfrac{1}{2}h\,k_2\big), & k_4 &= A(y_1)\big(Y + h\,k_3\big), \end{aligned} \qquad Y \leftarrow Y + \frac{h}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big), \tag{19}$$

with $y_m = \frac{1}{2}(y_0 + y_1)$ the step midpoint and $A(\cdot)$ the pointwise matrix of §10.

Re-orthonormalisation. The three solutions decay at very different rates, so after only a short march the fast-decaying columns are numerically swamped by the slow one and the basis collapses toward a single direction (a parasitic-growth / stiffness pathology well known in shooting). To suppress it, after *every* RK4 step the three columns are re-orthonormalised by a QR (Gram–Schmidt) step,

$$Y \;\longleftarrow\; Q, \qquad \text{where} \quad Y = QR \;\; (Q \in \mathbb{C}^{6\times 3} \text{ orthonormal}). \tag{20}$$

Only the orthonormal factor Q is retained; the upper-triangular R (which merely records the discarded scaling) is thrown away. This keeps the three columns well-conditioned all the way to the wall without changing the *space* they span — and it is that span, not the individual column magnitudes, that the eigenvalue condition below depends on.

In plain words. *QR after every step is a re-basing: it rotates the three tangled solutions back to a clean orthonormal frame while preserving the subspace they represent. The eigenvalue test cares only about which three-dimensional subspace of $\mathbb{C}^6$ arrives at the wall, so this scaling is harmless yet essential for accuracy.*

13 The eigenvalue condition and the wall matrix

What problem is solved here. Detect the special c for which a non-trivial disturbance satisfies all three wall conditions (17) at once. **Eigenvalue:** the phase speed c that makes the wall matrix singular.

At the wall the integrated basis is a 6×3 matrix $Y(0)$; any admissible disturbance is a combination $Y(0)\,a$ of its three columns with coefficients $a \in \mathbb{C}^3$. The three wall conditions (17) ask that rows $1, 3, 5$ of that combination vanish. Collecting those rows gives the 3×3 **wall matrix** (function `ozgen_temporal_wall_matrix_2d`)

$$M(c) = \begin{bmatrix} Y_1(0) \\ Y_3(0) \\ Y_5(0) \end{bmatrix} \in \mathbb{C}^{3\times 3}, \qquad \text{wall conditions:} \quad M(c)\,a = 0. \tag{21}$$

A *non-trivial* coefficient vector $a \neq 0$ exists only when $M(c)$ is **singular**. Rather than test a determinant (which is poorly scaled after the repeated QR re-scaling), the code monitors the *smallest singular value* of $M(c)$ (function `ozgen_temporal_sigma_min_2d`),

$$\sigma_{\min}(c) = \min \ \mathrm{svd}\big(M(c)\big), \qquad \boxed{\sigma_{\min}(c) \to 0 \iff c \text{ is an eigenvalue}}. \tag{22}$$

In plain words. *$\sigma_{\min}$ is a clean, always-non-negative "distance to singular." It is zero exactly when the three decaying solutions can be combined to meet all three wall conditions — that is, exactly when a genuine boundary-layer mode exists at that c. The eigenvalue search below just hunts for the c that drives this one real number to zero.*

14 The eigenvalue search

What problem is solved here. Locate the complex phase speed $c = c_r + \mathrm{i}c_i$ at which $\sigma_{\min}(c) = 0$, starting from a physics-based guess. This single c is the temporal eigenvalue, identical to the one Part A returns.

Because c is complex, driving $\sigma_{\min}(c) \to 0$ is a two-dimensional real minimisation over (c_r, c_i). `pyMack` runs a Nelder–Mead simplex (`scipy.optimize.minimize`, function `solve_ozgen_temporal_mode_2d_sigma_min`) on the objective $\sigma_{\min}$, from a physics-based initial guess c_{guess} (e.g. the second-mode value $c_r \approx 1 - 1/M_e$). The search is confined to the physical strip

$$c_r \in (0,\, 1.2), \qquad c_i \in (-0.2,\, 0.2), \tag{23}$$

enforced by returning a large penalty (10^6) whenever a trial leaves the box, and it converges to the simplex tolerances

$$\texttt{xatol} = 10^{-7} \text{ (in } c), \qquad \texttt{fatol} = 10^{-8} \text{ (in } \sigma_{\min}), \tag{24}$$

within at most ~ 120 iterations. Each objective evaluation runs the *entire* pipeline of §§11–13 (Figure 12): build the freestream decay basis, RK4-integrate with QR re-orthonormalisation to the wall, form $M(c)$, and take $\sigma_{\min}$. The converged minimiser $c = c_r + \mathrm{i}c_i$ is the eigenvalue, and the temporal growth rate follows exactly as in Part A, $\omega_i = \alpha c_i$. The table below is the same five-stage loop as the figure, mapped to functions.

step	operation	function
1	build the 6×6 pointwise matrix $A(y)$ from the mean flow	`ozgen_first_order_matrix_2d`
2	eigen-decompose $A(y_{\max})$; keep the 3 decaying modes; QR to a 6×3 basis	`ozgen_freestream_decay_basis_2d`
3	RK4-march the 6×3 basis $y_{\max} \to 0$, QR after every step	`integrate_ozgen_bounded_basis_2d`
4	collect wall rows (X_1, X_3, X_5) into $M(c)$; take $\sigma_{\min}(c)$	`ozgen_temporal_wall_matrix_2d`, `ozgen_temporal_sigma_min_2d`
5	Nelder–Mead over (c_r, c_i) until $\sigma_{\min} \to 0$; return c	`solve_ozgen_temporal_mode_2d_sigma_min`

15 Spectral versus shooting: the same eigenvalue, two temperaments

The two solvers answer the identical question — "for this real α, what complex c admits a non-trivial disturbance decaying at the freestream and vanishing at the wall?" — and return the same physical phase speed c, but they trade off differently:

	Spectral (Part A)	**Shooting (Part B)**
core object	one $4n \times 4n$ pair (A, B)	one 6×6 matrix $A(y)$, evaluated pointwise
what it returns	*all* $4n$ eigenvalues at once	*one* eigenvalue near a guess
freestream decay	identity rows on the edge nodes	keep the 3 decaying eigenvectors of $A(y_{\max})$
wall conditions	identity row-replacement	$M(c)$ singular, $\sigma_{\min}(c) \to 0$
how the answer is found	dense QZ / shift–invert	RK4 march + QR, then Nelder–Mead on $\sigma_{\min}$
memory / cost	large dense matrix, $O((4n)^3)$	tiny system, cheap per evaluation
best used for	surveying the whole spectrum; sweeping neutral curves and N-factors	pinpointing one mode to high accuracy from a good guess

In plain words. *Think of the spectral method as a* census *— it enumerates every eigenvalue on the grid, which is exactly what you want when tracing a neutral curve or hunting an unknown branch — and shooting as a* sniper*: given a good guess it drives one mode to machine-tolerance accuracy with almost no memory and no large matrix. Used together, the spectral solver locates a mode and shooting refines and cross-checks it; their agreement on c is a strong verification that the eigenvalue is real physics, not a discretisation artefact.*

Code map (Part B). first-order matrix, decay basis, integration, wall matrix, singular value, and search: all in `pymack/temporal_shooting.py`; base-flow sampling shared via `pymack/mack_shooting.py` (`_sample_scaled_baseflow`). References: Ozgen & Kircali (2008), *Theor. Comput. Fluid Dyn.*, Eqs. 22–28; Mack (1984), AGARD-R-709.